```
import networkx as nx
import random as rd
import copy
import matplotlib.pyplot as plt
import numpy as np
from tqdm import tqdm
import math
```

Search on graphs

Mostrar código

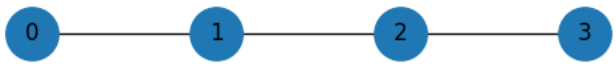
TreeSampler class

Mostrar código

Our initial verification test uses the path 0 -> 1 -> 2

```
e = [(0,1), (1,2), (2,3)] #list of edges
G = nx.Graph(e)

pos = {0: [0,0], 1: [1,0], 2: [2,0], 3: [3,0]}
fig = plt.figure(1, figsize=(5, 1))
nx.draw(G, with_labels= True, pos=pos, node_size= 800)
```



Setting attribute to account for the real epidemics in the graph

```
attrs = {0: {"inf_time": 1, "status": 'recovered'}, 1: {"inf_time": 2, "status": 'recovered'},
         2: {"inf_time": 3, "status": 'recovered'}, 3: {"inf_time": math.inf, "status": 'susceptible'}}
nx.set_node_attributes(G, attrs)
```

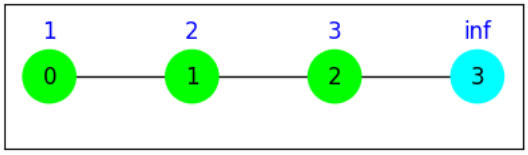
Visualizing the graph with their respectives infection times

```
color_state_map = {'recovered': 'lime', 'susceptible': 'cyan'}
node_color = [color_state_map[node[1]['status']] for node in G.nodes(data=True)]

infection_times = nx.get_node_attributes(G, "inf_time")
state_pos = {n: (x, y + 0.035) for n, (x,y) in pos.items()}

fig = plt.figure(1, figsize=(5, 1.4))
nx.draw_networkx(G, pos, node_size = 800, node_color = node_color,)

nx.draw_networkx_labels(G, state_pos, labels= infection_times, font_color='blue')
plt.show()
```



Partial information available

```
#First we make a copy of G with the real infection times
G_real = copy.deepcopy(G)
```

```
#We exclude the information for nodes 2 and 3
nx.set_node_attributes(G, {2: {"inf_time": math.inf, "status": None}, 3: {"inf_time": math.inf, "status": None}}
```

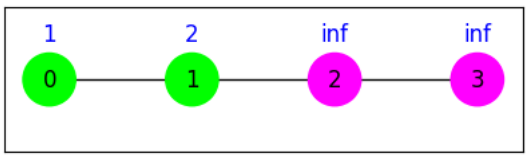
Visualizing the graph with only the partial information available

```
color_state_map = {'recovered': 'lime', 'susceptible': 'cyan', None: 'magenta'}
node_color = [color_state_map[node[1]['status']] for node in G.nodes(data=True)]

infection_times = nx.get_node_attributes(G, "inf_time")
state_pos = {n: (x, y + 0.035) for n, (x,y) in pos.items()}

fig = plt.figure(1, figsize=(5, 1.4))
nx.draw_networkx(G, pos, node_size = 800, node_color = node_color,)

nx.draw_networkx_labels(G, state_pos, labels= infection_times, font_color='blue')
plt.show()
```




```
#Copying the partial information to use different sample sizes
G_10000 = copy.deepcopy(G)
G_100000 = copy.deepcopy(G)
```

Set an initial feasible tree for G given the observed values

```
# The feasible tree data structure is a list of lists
# Each element in T_initial is a path in the tree
T_initial = feasible_tree(G, [0,1], flag=1)
print(T_initial)
```

```
[[0], [1, 0]]
```

Returns the frequency (in %) of the nodes in the sampling

Mostrar código

Start the sampling algorithm

(at each run we plot information about the current tree, choosed operations and so forth in order to manually verify the correctness of the outputs, see output.txt file)

```
infected_nodes = [0,1]
samplings_number = 1000

#Initialize class
sampler_1000 = TreeSampler(G, T_initial, infected_nodes, seed=10, flag=1)

#Run
sampling = sampler_1000.run(n_iterations=samplings_number)
print(f"Real infection times: {nx.get_node_attributes(G_real, "inf_time")}")
print("-----------------------------------------------------------------------------------------------")
print(f"Frequency of nodes: {nodes_proportion(G, sampling)}")
```

```
Sampling trees: 100%|██████████| 1000/1000 [00:00<00:00, 4636.30it/s]
Final Acceptance Rate: 53.70%
Real infection times: {0: 1, 1: 2, 2: 3, 3: inf}
-----------------------------------------------------------------------------------------------
Frequency of nodes: {0: 1.0, 1: 1.0, 2: 0.24075924075924077, 3: 0.04295704295704296}
```

```
samplings_number = 10000

sampler_10000 = TreeSampler(G_10000, T_initial, infected_nodes, seed=10,flag=1)

#Run
sampling = sampler_1000.run(n_iterations=samplings_number)
print(f"Real infection times: {nx.get_node_attributes(G_real, "inf_time")}")
print("-----------------------------------------------------------------------------------------------")
print(f"Frequency of nodes: {nodes_proportion(G_10000, sampling)}")
```

```
Sampling trees: 100%|██████████| 10000/10000 [00:03<00:00, 2945.47it/s]
Final Acceptance Rate: 52.09%
Real infection times: {0: 1, 1: 2, 2: 3, 3: inf}
```

```
----------------------------------------------------------------------------------------------------
Frequency of nodes: {0: 1.0, 1: 1.0, 2: 0.2852467957458413, 3: 0.0684483228797382}
```

```
samplings_number = 100000

sampler_100000 = TreeSampler(G_100000, T_initial, infected_nodes,seed=10, flag=1)

#Run
sampling = sampler_100000.run(n_iterations=samplings_number)
print(f"Real infection times: {nx.get_node_attributes(G_real, "inf_time")}")
print("----------------------------------------------------------------------------------------------------")
print(f"Frequency of nodes: {nodes_proportion(G_100000, sampling)}")
```

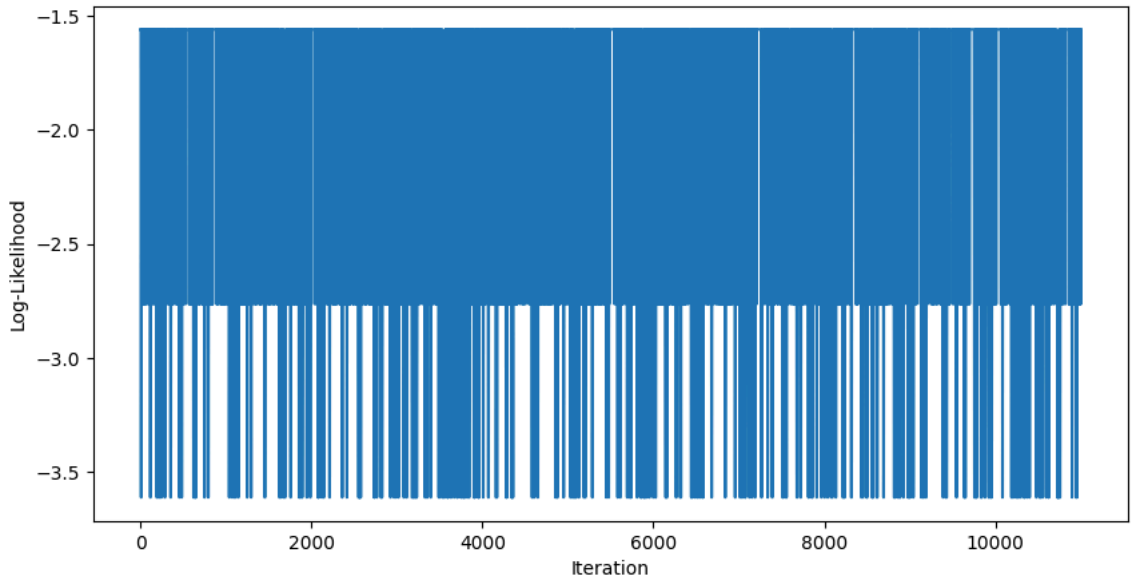
```
Sampling trees: 100%|██████████| 100000/100000 [00:15<00:00, 6545.82it/s]

Final Acceptance Rate: 52.09%
Real infection times: {0: 1, 1: 2, 2: 3, 3: inf}
----------------------------------------------------------------------------------------------------
Frequency of nodes: {0: 1.0, 1: 1.0, 2: 0.2827771722282777, 3: 0.07041929580704193}
```

Plots (visualizing results)

Trace plot (still in progress)

```
sampler_1000._trace_plot_log_likelihood()
```



Next step: Convergence diagnostics

ref: https://arxiv.org/pdf/1909.11827